What is Fine-Tuning?

Fine-tuning involves taking a pre-trained language model (which has already learned a vast amount of general knowledge and language patterns from a diverse dataset) and further training it on a smaller, task-specific, or domain-specific dataset.This process “adjusts” the model’s internal parameters (weights) to better understand and generate text aligned with the nuances, jargon, style, or specific tasks of the new data.

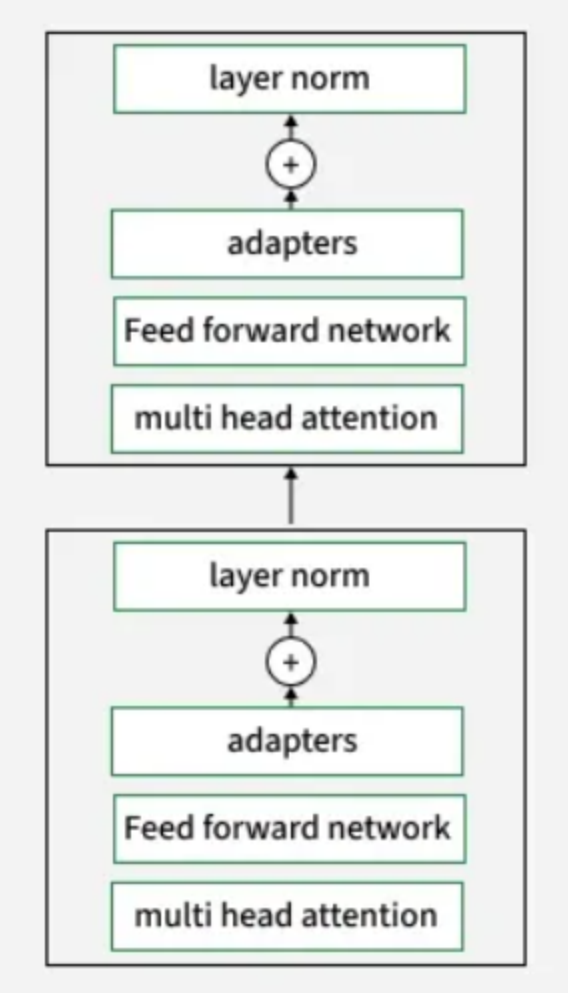
Traditional Fine-Tuning

In its original form, fine-tuning involves updating all, or a significant portion, of a pre-trained model’s parameters.For models with hundreds of millions or even billions of parameters, this is a computationally intensive and resource-demanding process. It requires substantial GPU power, memory, and time, making it less practical for frequent updates or for users with limited hardware. The output is a new, specialized version of the entire model.

LoRA (Low-Rank Adaptation)

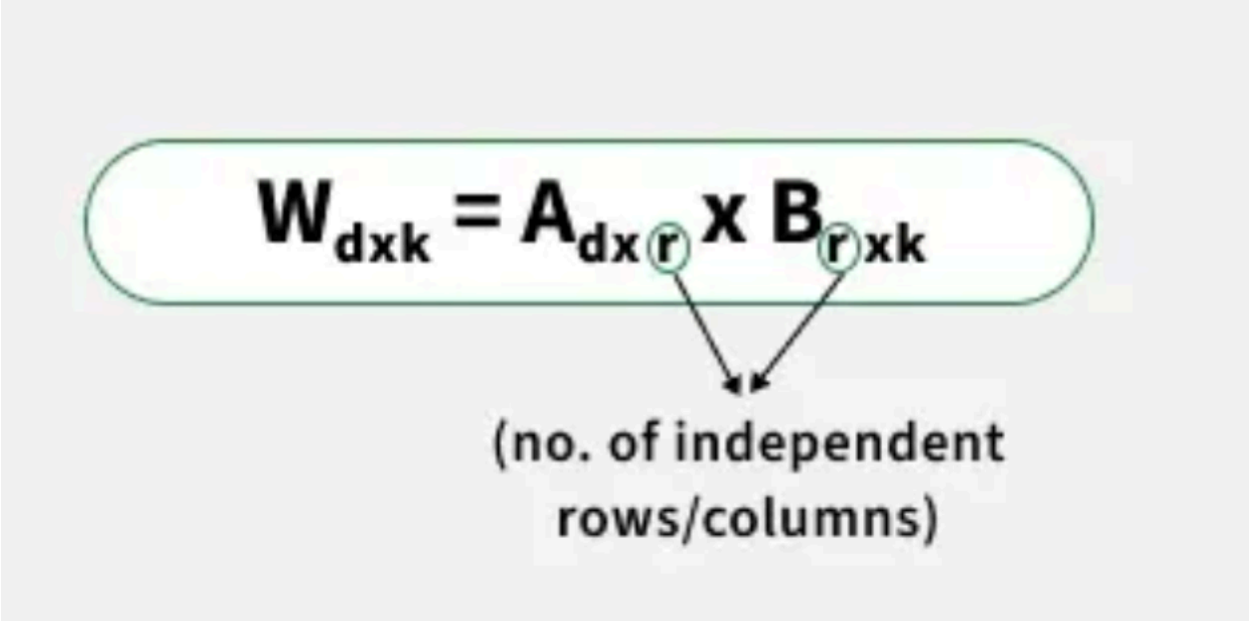
How LoRA Works

The below image shows a transformer block architecture where **adapters** are inserted after the feed-forward network and before layer normalization. In LoRA, these adapters are implemented as low-rank matrices. During fine-tuning, only these adapter parameters are updated, while the core model weights (multi-head attention, feed-forward network, etc.) stay fixed.



Adapter Layer in LoRA

- **Parameter Update: Only the adapters (the low-rank matrices) are updated during training, greatly reducing the number of trainable parameters.**

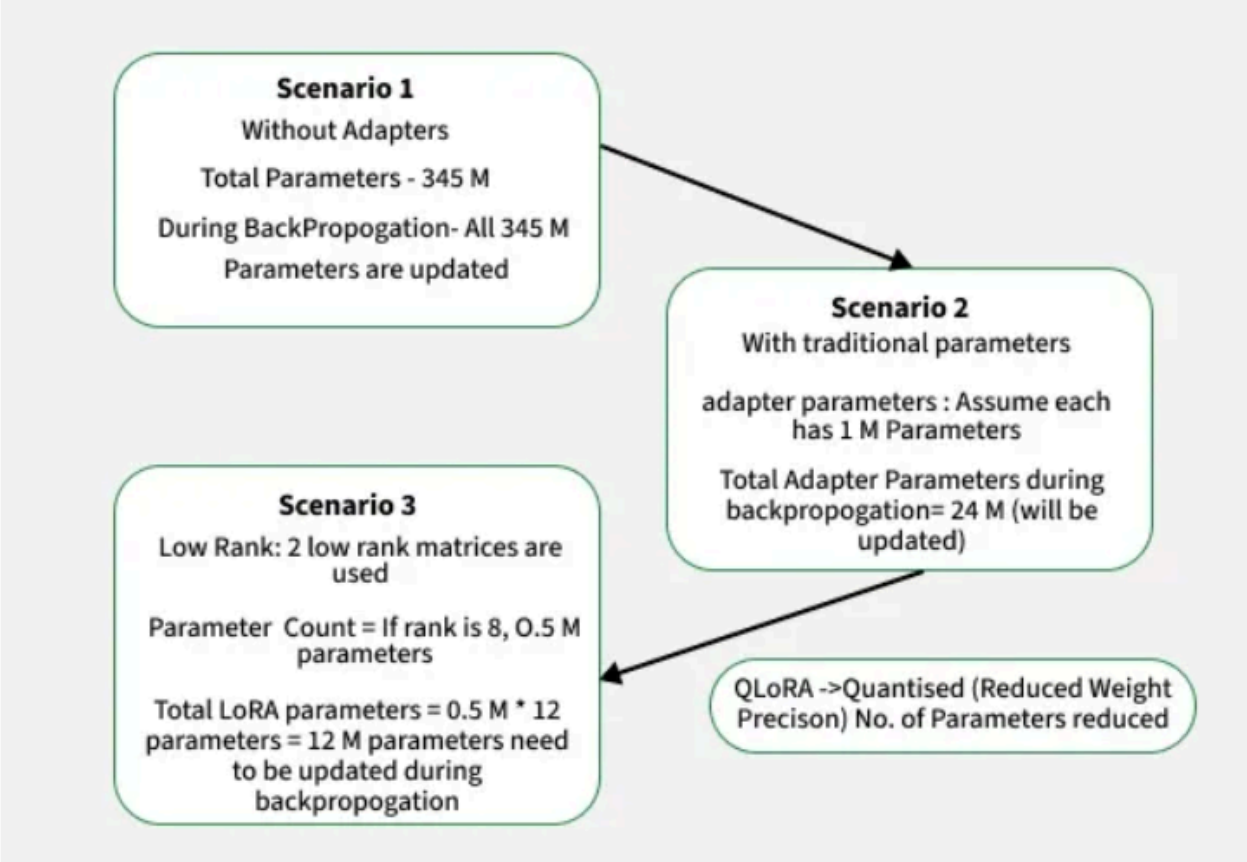


QLoRA (Quantized LoRA)

QLoRA(Quantized LoRA) works by loading the base language model in a highly compressed 4-bit quantized format, drastically reducing memory usage, while training small LoRA adapters in higher precision. During fine-tuning, only these adapters are updated, compensating for any quantization errors and preserving model performance. This approach allows you to efficiently fine-tune massive models on standard GPUs, combining aggressive memory savings with the parameter efficiency of LoRA and maintaining competitive results.

Training Process:

- **The pretrained model is loaded with quantized 4-bit weights.**
- **Only the LoRA adapters are updated during training.**
- **Libraries like BitsAndBytes (for quantization) and PEFT (for LoRA) are used together for implementation.**
- **Example code involves setting up quantization configs, enabling gradient checkpointing, and applying LoRA adapters to target modules (e.g., query, key, value layers).**



- **Scenario 1: Full finetuning All 345M model parameters are updated.**
- **Scenario 2: Adapter tuning Only 24M adapter parameters are updated (1M per adapter × 24).**
- **Scenario 3: LoRA Just 12M low-rank adapter parameters are updated (0.5M × 12 × 2).**
- **QLoRA: Like LoRA, but with quantized (lower precision) weights, further reducing parameter count and memory use.**